

IMPLEMENTASI RESTFUL & GRAPHQL API PADA SISTEM WISATAKU

Nama : Ahmad Damanhuri
NIM : SI21240001
Mata Kuliah : Teknologi Web Service
Program Studi : Sistem Informasi
Semester : 5

BAB I

PENDAHULUAN

1.1 Latar Belakang

Web Service merupakan teknologi yang memungkinkan aplikasi saling berkomunikasi dan bertukar data melalui jaringan. Dalam pengembangan aplikasi modern, API digunakan sebagai penghubung antara aplikasi dengan layanan atau database sehingga data dapat diakses secara terstruktur.

Pada praktikum Teknologi Web Service, dikembangkan sebuah sistem bernama WisataKu, yaitu sistem informasi dan reservasi destinasi wisata. Salah satu bagian penting dalam pengembangan sistem tersebut adalah menyediakan API yang dapat digunakan untuk mengakses dan mengelola data destinasi wisata.

Pada praktikum ini dilakukan implementasi dua pendekatan API, yaitu RESTful API dan GraphQL API. RESTful API digunakan untuk menyediakan operasi CRUD (*Create, Read, Update, Delete*) terhadap data destinasi wisata. Implementasi tersebut menggunakan NestJS, Prisma ORM, dan MariaDB sebagai database.

Selanjutnya, GraphQL digunakan sebagai alternatif dalam mengakses data destinasi dengan menyediakan query yang dapat menentukan data apa saja yang ingin diambil oleh client.

Dengan implementasi tersebut, sistem WisataKu memiliki layanan API yang dapat diuji dan digunakan untuk mengakses data destinasi melalui REST maupun GraphQL.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah:

1. Mengimplementasikan RESTful API menggunakan NestJS.
2. Mengimplementasikan operasi CRUD pada data destinasi wisata.

3. Menghubungkan aplikasi NestJS dengan database MariaDB menggunakan Prisma ORM.
4. Mengimplementasikan GraphQL API menggunakan NestJS dan Apollo.
5. Membuat GraphQL Type dan Resolver untuk data destinasi.
6. Menguji RESTful API menggunakan Postman.
7. Menguji GraphQL API menggunakan GraphiQL.
8. Memastikan API dapat mengambil data dari database dan memberikan response yang sesuai.

1.3 Ruang Lingkup Praktikum

Implementasi pada praktikum ini berfokus pada data Destinasi pada sistem WisataKu.

Data destinasi yang digunakan memiliki atribut:

Atribut	Keterangan
id	Identitas unik destinasi
nama	Nama destinasi wisata
kategori	Kategori destinasi
lokasi	Lokasi destinasi
deskripsi	Deskripsi destinasi
hargaTiket	Harga tiket masuk
ratingRata	Nilai rata-rata rating

Implementasi praktikum terdiri atas dua bagian:

- Implementasi RESTful API, menggunakan Prisma dan MariaDB.
- Implementasi GraphQL API, menggunakan NestJS dan Apollo.

BAB II

IMPLEMENTASI RESTFUL API

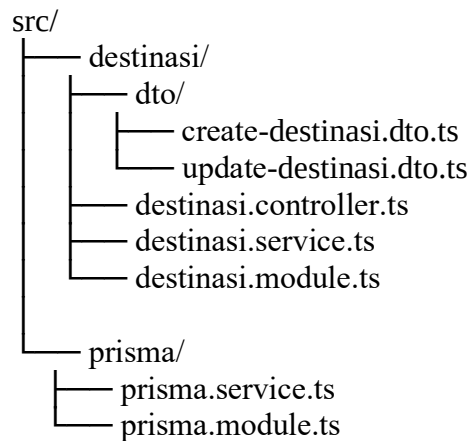
2.1 Teknologi yang Digunakan

Implementasi RESTful API menggunakan beberapa teknologi berikut:

Teknologi	Fungsi
TypeScript	Bahasa pemrograman
Node.js	Runtime JavaScript
NestJS	Framework backend
Prisma	ORM untuk akses database
MariaDB	Database relasional
Class Validator	Validasi data
Swagger	Dokumentasi API
Postman	Pengujian API

2.2 Struktur RESTful API

Struktur utama implementasi RESTful API adalah:



DestinasiController bertugas menerima request dari client, sedangkan DestinasiService menangani proses pengolahan data melalui Prisma.

2.3 Implementasi Database dengan Prisma

Database yang digunakan adalah **MariaDB** dengan nama database:

wisataku

Model Destinasi pada Prisma memiliki struktur:

```
model Destinasi {  
  id          Int      @id @default(autoincrement())  
  nama        String  
  kategori    String  
  lokasi      String  
  deskripsi   String  
  hargaTiket  Int  
  ratingRata  Float  
}
```

Prisma digunakan sebagai ORM untuk menghubungkan aplikasi NestJS dengan database MariaDB.

Alur pengaksesan data:

```
Client  
↓  
DestinasiController  
↓  
DestinasiService  
↓  
PrismaService  
↓  
MariaDB
```

2.4 Endpoint RESTful API

Endpoint yang diimplementasikan adalah:

Method	Endpoint	Fungsi
GET	/destinasi	Mengambil seluruh destinasi
GET	/destinasi/:id	Mengambil destinasi berdasarkan ID
POST	/destinasi	Menambahkan destinasi
PATCH	/destinasi/:id	Memperbarui destinasi
DELETE	/destinasi/:id	Menghapus destinasi

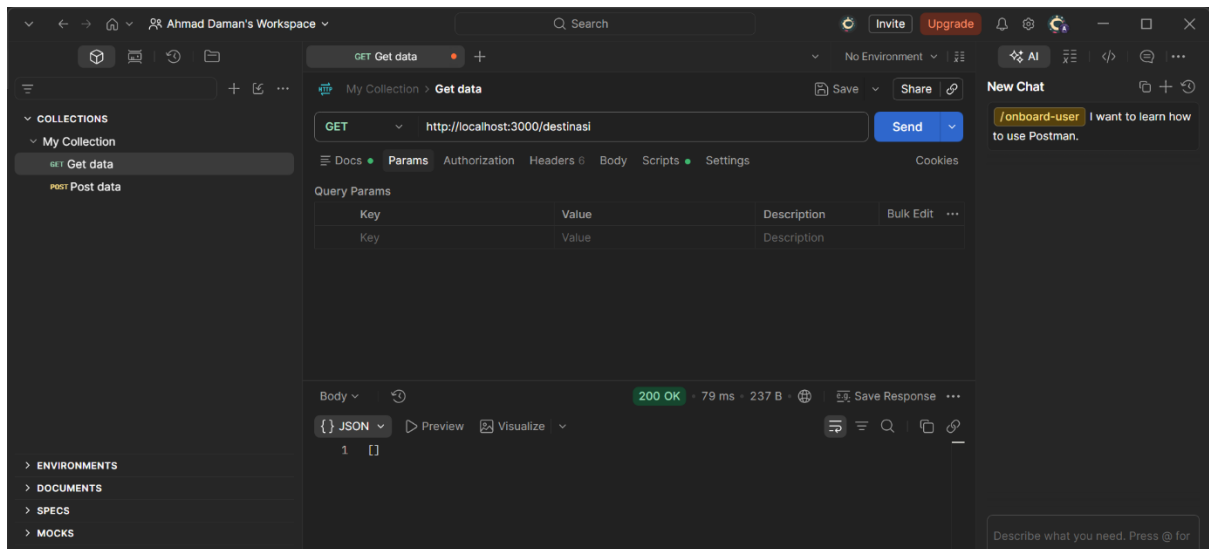
2.5 Pengujian GET /destinasi

Pengujian dilakukan menggunakan Postman.

Request:

GET http://localhost:3000/destinasi

Pada pengujian awal, database belum memiliki data sehingga response yang diperoleh:



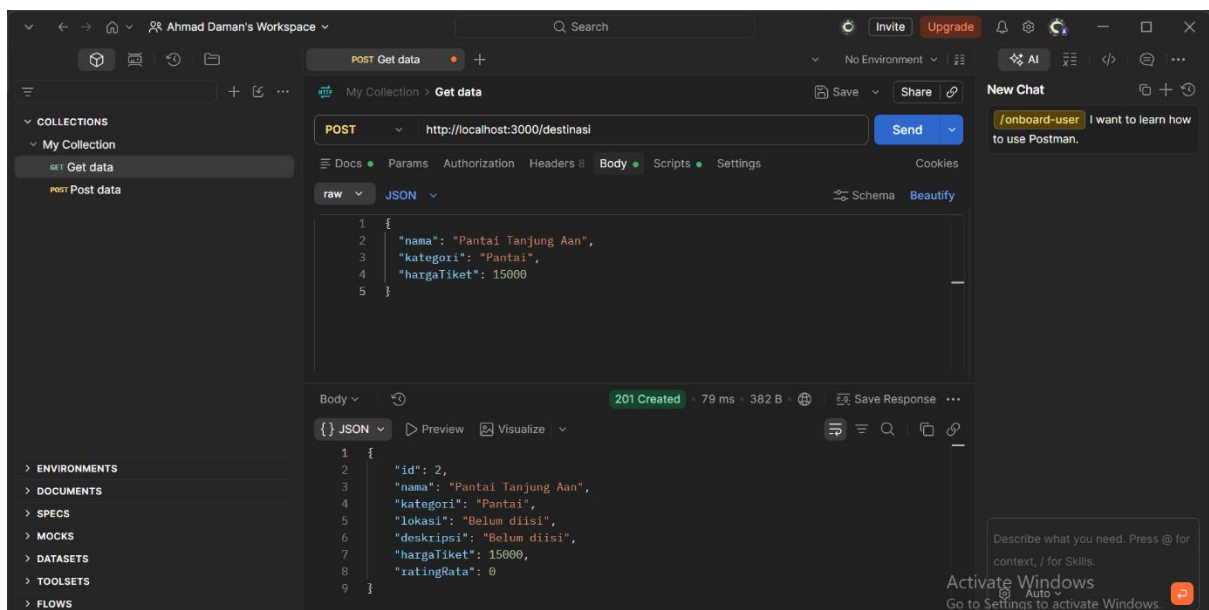
Gambar 2.1. Pengujian GET /destinasi

Status:

200 OK

2.6 Pengujian POST /destinasi

Data yang dikirim:



Gambar 2.2. Pengujian POST /destinasi

Status response:

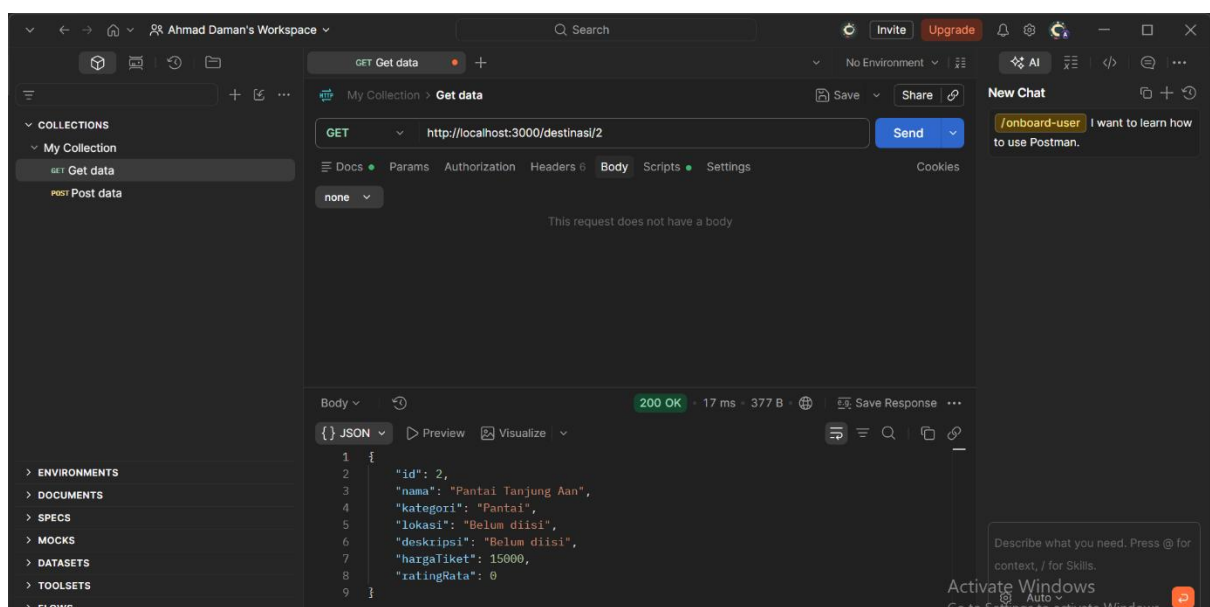
201 Created

Data tersebut berhasil disimpan ke database MariaDB.

2.7 Pengujian GET /destinasi/:id

Pengujian digunakan untuk mengambil data destinasi berdasarkan ID.

Request:



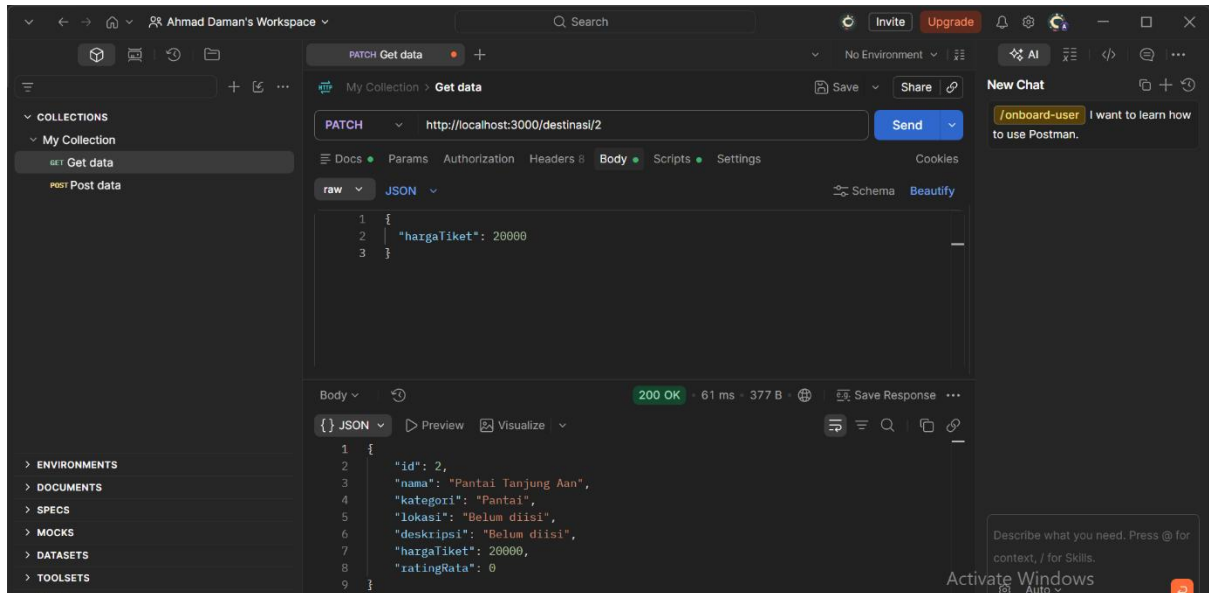
Gambar 2.3. Pengujian GET /destinasi/:id

Response menunjukkan data destinasi berdasarkan ID yang diminta.

2.8 Pengujian PATCH /destinasi/:id

Pengujian PATCH digunakan untuk memperbarui data destinasi.

Request body:



Gambar 2.4. Pengujian PATCH /destinasi/:id

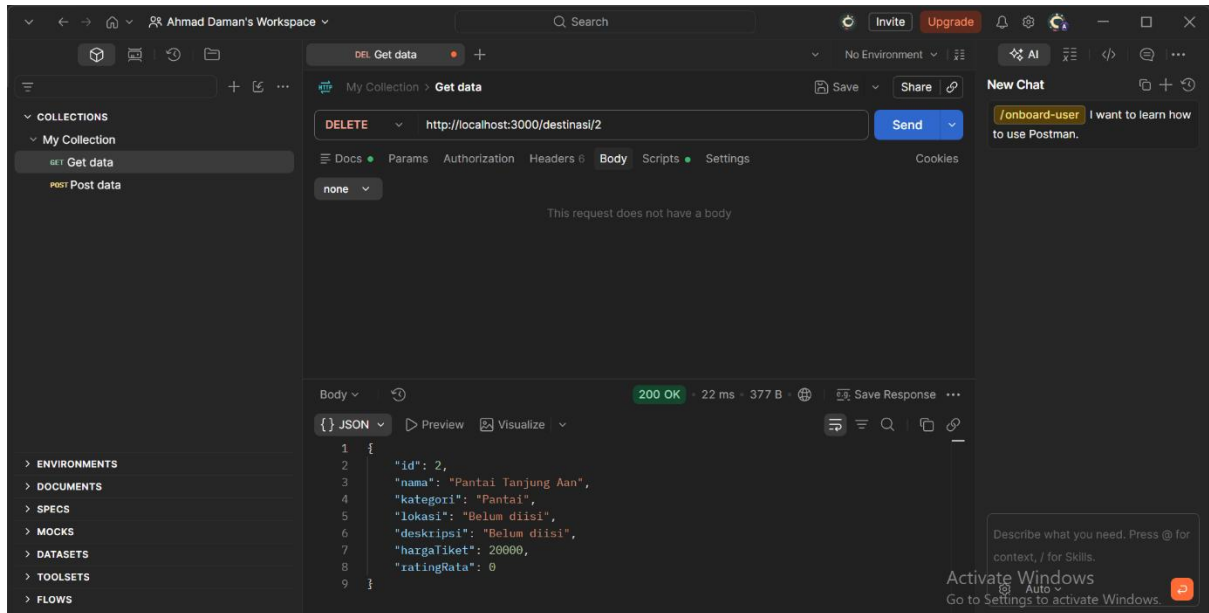
Status response:

200 OK

2.9 Pengujian DELETE /destinasi/:id

Pengujian DELETE dilakukan untuk menghapus data destinasi.

Request:



Gambar 2.5. Pengujian DELETE /destinasi/:id

200 OK

Setelah penghapusan, dilakukan kembali GET /destinasi untuk memastikan data tersebut sudah tidak tersedia.

BAB III

IMPLEMENTASI GRAPHQL API

3.1 Teknologi GraphQL

Pada implementasi GraphQL digunakan:

Teknologi	Fungsi
@nestjs/graphql	Integrasi GraphQL dengan NestJS
@nestjs/apollo	Apollo GraphQL driver
@apollo/server	GraphQL server
graphql	Library GraphQL
GraphiQL	Pengujian query GraphQL

GraphQL digunakan sebagai pendekatan API alternatif selain RESTful API untuk mengakses data destinasi.

3.2 Konfigurasi GraphQL

GraphQL dikonfigurasi pada `app.module.ts` menggunakan `ApolloDriver`.

Konfigurasi yang digunakan:

```
GraphQLModule.forRoot<ApolloDriverConfig>({  
  driver: ApolloDriver,  
  autoSchemaFile: join(process.cwd(), 'src/schema.gql'),  
  playground: true,  
})
```

Konfigurasi tersebut membuat GraphQL berjalan pada endpoint:

<http://localhost:3000/graphql>

Schema GraphQL dibuat secara otomatis oleh NestJS melalui konfigurasi `autoSchemaFile`.

3.3 Implementasi GraphQL Type

GraphQL Type digunakan untuk mendefinisikan struktur data yang dapat dikembalikan oleh GraphQL.

File:

src/graphql/destinasi.type.ts

Type `DestinasiType` memiliki field:

id
nama
kategori
lokasi
deskripsi
hargaTiket
ratingRata

Dengan demikian, struktur data GraphQL sesuai dengan data destinasi yang terdapat pada database.

3.4 Implementasi GraphQL Resolver

Resolver digunakan sebagai penghubung antara query GraphQL dengan service aplikasi.

File:

src/graphql/destinasi.resolver.ts

Resolver menyediakan dua query:

Query destinasi

Digunakan untuk mengambil seluruh data destinasi.

Query destinasiById

Digunakan untuk mengambil satu destinasi berdasarkan ID.

Alur proses GraphQL adalah:

Client / GraphiQL



DestinasiResolver



DestinasiService



PrismaService



MariaDB

3.5 Schema GraphQL

Schema yang terbentuk secara otomatis menghasilkan struktur:

```
type DestinasiType {  
  id: Int!  
  nama: String!  
  kategori: String!  
  lokasi: String!  
  deskripsi: String!  
  hargaTiket: Int!  
  ratingRata: Float!  
}  
  
type Query {  
  destinasi: [DestinasiType!]!  
  destinasiById(id: Int!): DestinasiType!  
}
```

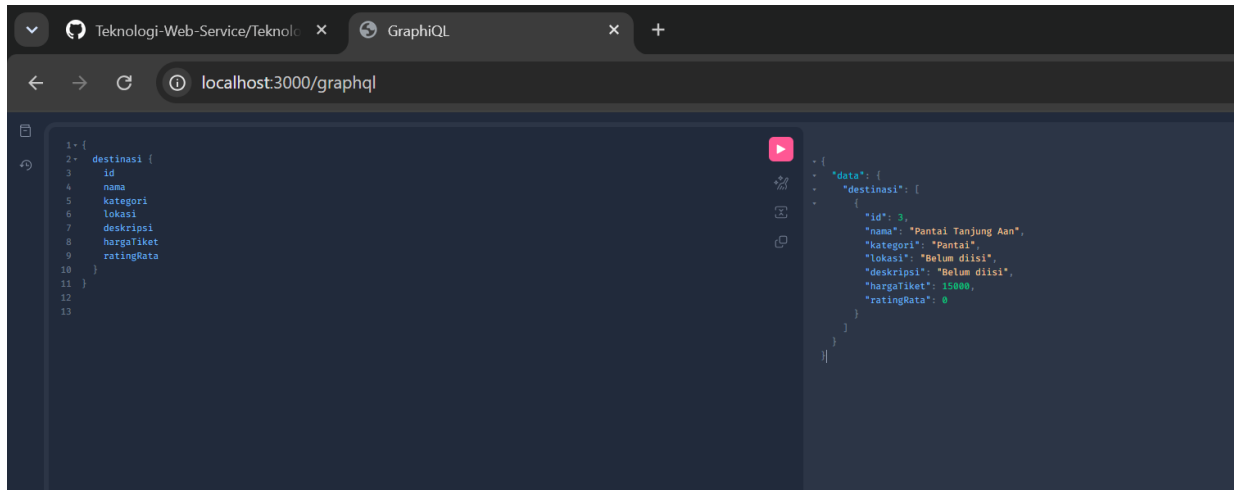
Schema tersebut menunjukkan bahwa GraphQL menyediakan dua operasi query untuk mengakses data destinasi.

BAB IV

PENGUJIAN GRAPHQL API

4.1 Pengujian Query destinasi

Pengujian dilakukan melalui GraphiQL pada:

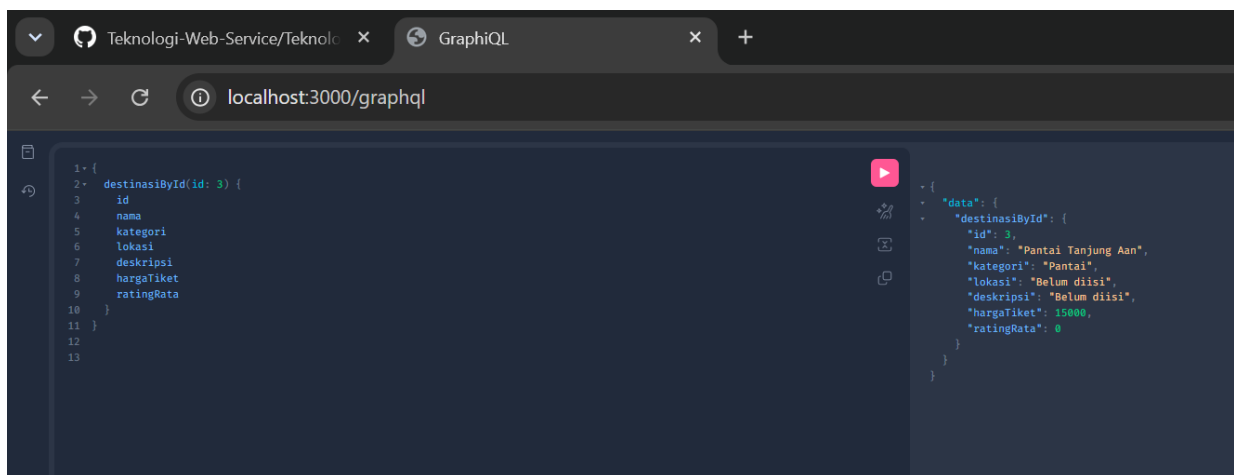


Gambar 4.1. Pengujian Query GraphQL destinasi

Hasil tersebut menunjukkan bahwa GraphQL berhasil mengambil data destinasi dari database melalui resolver dan service.

4.2 Pengujian Query destinasiById

Pengujian kedua dilakukan untuk mengambil satu data destinasi berdasarkan ID.



Gambar 4.2. Pengujian Query GraphQL destinasiById

BAB V

HASIL DAN PEMBAHASAN

5.1 Hasil Implementasi RESTful API

Berdasarkan pengujian yang dilakukan, RESTful API berhasil menjalankan operasi pengelolaan data destinasi.

Operasi yang berhasil diuji meliputi:

GET → Berhasil
POST → Berhasil
GET /:id → Berhasil
PATCH /:id → Berhasil
DELETE /:id → Berhasil

RESTful API juga berhasil terhubung dengan database MariaDB melalui Prisma.

5.2 Hasil Implementasi GraphQL API

GraphQL berhasil dijalankan pada endpoint:

<http://localhost:3000/graphql>

Dua query yang diimplementasikan berhasil dijalankan:

destinasi → berhasil mengambil daftar destinasi
destinasiById → berhasil mengambil destinasi berdasarkan ID

Hasil response menunjukkan bahwa data yang diperoleh melalui GraphQL berasal dari database MariaDB melalui DestinasiService dan Prisma.

BAB VI

KESIMPULAN

Berdasarkan hasil implementasi dan pengujian, dapat disimpulkan bahwa praktikum Implementasi RESTful & GraphQL API pada Sistem WisataKu telah berhasil dilakukan.

RESTful API berhasil diimplementasikan menggunakan NestJS dengan operasi pengelolaan data destinasi berupa GET, POST, PATCH, dan DELETE. API tersebut terhubung dengan database MariaDB menggunakan Prisma ORM.

Selain itu, GraphQL API berhasil diimplementasikan menggunakan NestJS, Apollo, dan GraphQL. Implementasi GraphQL menyediakan query destinasi untuk mengambil daftar destinasi dan destinasiById untuk mengambil data destinasi berdasarkan ID.

Hasil pengujian menunjukkan bahwa RESTful API dan GraphQL API dapat berjalan serta mengambil data melalui Prisma dari database MariaDB. Dengan demikian, implementasi API pada sistem WisataKu telah berhasil dilakukan sesuai dengan ruang lingkup praktikum.